

Session 2: From Concept to Code! Building Your First Agent

Welcome back to "Mastering Agentic AI: From Concept to Code"! In our last session, we reinforced the fundamental definition of an AI agent and set the stage for understanding how intelligence, even simple intelligence, drives autonomous behavior.

Today, we're rolling up our sleeves for our first hands-on project: building a simple **Traffic Light Agent** using Python. This project will serve as your very first "agent blueprint," demonstrating the core components of an agent in a straightforward, rule-based system.

Session Goal:

By the end of this session, you will be able to:

- Understand the concept of a "state machine" as a basic building block for agents.
- Implement simple, rule-based agent logic using Python's if-elif-else structure.
- Observe the fundamental "perceive-decide-act" cycle in a practical example.
- Write your first functional Python code representing an agent.

Why a "Traffic Light Agent"?

A traffic light might seem simple, but it's an excellent analogy for an agent because it perfectly embodies the core agentic loop:

- **Perception (Sensors):** It "perceives" time passing (its internal timer).
- **Decision (Agent Program):** Based on the perceived time and its current state (e.g., "Red" for 30 seconds), it "decides" to change.
- **Action (Actuators):** It "acts" by changing its light color (e.g., from Red to Green).
- **Environment:** The intersection itself, where the light color dictates traffic flow.

This project allows us to focus on basic agent logic without the complexities of advanced AI, making it perfect for freshers to grasp the foundational concepts.

Core Concepts Covered Today:

1. State Machine:

- Imagine a system that can be in only one of a finite number of "states" at any given time. A traffic light, for example, can be Red, Yellow, or Green – these are its states.
- A state machine defines how the system transitions from one state to another based on specific conditions or events.
- This is a fundamental concept in computing and AI, providing a structured way to manage behavior.

2. Rule-Based System:

- In a rule-based system, the agent's decisions are driven by explicit "if-then" rules.
- For our traffic light, a rule might be: "IF current state is GREEN AND 30 seconds have passed, THEN change state to YELLOW."
- This is in contrast to more complex AI like Machine Learning, where rules are learned implicitly from data. Here, we define them directly.

3. Basic Agent Logic (Perceive-Decide-Act):

- Every intelligent agent, no matter how complex, follows this fundamental cycle.
- Our traffic light agent will continuously:
 - **Perceive:** Check its internal timer.
 - **Decide:** Use if-elif-else rules to determine if a state change is needed.
 - **Act:** Update its current state and "display" the new light color.

Prerequisites:

- **Python Installed:** Ensure you have a working Python installation (preferably Python 3.8+). You can download it from python.org.
- **Basic Python Knowledge:** Familiarity with variables, if-else statements, while loops, and printing to the console.
- **Text Editor/IDE:** A text editor like VS Code, Sublime Text, or an IDE like PyCharm to write and run your Python code.

Step-by-Step Code Walkthrough:

We'll build our `traffic_light_agent.py` file together.

1. **Import time:** We'll use the `time.sleep()` function to pause the execution and simulate the passage of time, making our traffic light changes visible.
2. **Define States (Constants):** We'll use simple strings to represent the different colors/states of our traffic light. Using constants makes the code more readable and maintainable.

```
# Define the possible states of the traffic light
RED = "RED"
YELLOW = "YELLOW"
GREEN = "GREEN"
```

3. **Initial State and Timer:** Our agent needs a starting point and a way to track how

```
long it's been in the current state.  
current_light = RED  
time_in_current_state = 0
```

4. **Simulation Loop:** The heart of our agent will be a while True loop, allowing it to run continuously. Inside this loop, the "perceive-decide-act" cycle will happen repeatedly.

```
while True:
```

```
    # 1. Perceive: Increment the internal timer  
    time_in_current_state += 1
```

```
    # 2. Decide (Agent Program/Logic): Apply rules based on current state and  
    time
```

```
    # ... (logic goes here) ...
```

```
    # 3. Act: Print the current state and wait  
    print(f"Traffic Light is: {current_light} (Time in state: {time_in_current_state}  
seconds)")  
    time.sleep(1) # Simulate 1 second passing
```

5. **Implementing Transitions (if-elif-else):** This is where our rule-based logic comes in. We'll check the current_light and time_in_current_state to determine if a change is needed.

```
if current_light == RED and time_in_current_state >= 10: # Stay RED for 10  
seconds
```

```
    current_light = GREEN
```

```
    time_in_current_state = 0 # Reset timer for the new state
```

```
elif current_light == GREEN and time_in_current_state >= 15: # Stay GREEN for 15  
seconds
```

```
    current_light = YELLOW
```

```
    time_in_current_state = 0
```

```
elif current_light == YELLOW and time_in_current_state >= 5: # Stay YELLOW for 5  
seconds
```

```
    current_light = RED
```

```
    time_in_current_state = 0
```

- Notice how after each state change, we **reset time_in_current_state to 0**. This is crucial because the timer is specific to how long the agent has been in its *current* state.

Let's put it all together in the complete code.

code:

```
import time
```

```
# --- Agent Environment Configuration ---
```

```
# Define the possible states (colors) of the traffic light.
```

```
# Using constants makes the code more readable and less prone to typos.
```

```
LIGHT_RED = "RED"
```

```
LIGHT_YELLOW = "YELLOW"
```

```
LIGHT_GREEN = "GREEN"
```

```
# Define the duration (in simulated seconds) for each state.
```

```
# These are the rules governing the traffic light's behavior.
```

```
DURATION_RED = 10
```

```
DURATION_GREEN = 15
```

```
DURATION_YELLOW = 5
```

```
# --- Agent's Internal State ---
```

```
# Initialize the traffic light's starting state.
```

```
current_light_state = LIGHT_RED
```

```

# Initialize a timer to track how long the light has been in its current state.

time_in_current_state = 0

# --- Agent Program (The main logic loop) ---

def run_traffic_light_agent():
    """
    Simulates a simple traffic light agent that cycles through states
    (Red, Green, Yellow) based on predefined durations.
    This demonstrates basic perceive-decide-act agent logic.
    """

    global current_light_state, time_in_current_state

    print("--- Traffic Light Agent Started ---")

    # The main agent loop: runs continuously, simulating the environment.
    while True:

        # 1. Perceive (Simulated Sensor):

        # The agent "perceives" the passage of time by incrementing its internal timer.
        time_in_current_state += 1

        # 2. Decide (Agent Program/Logic):

        # The agent applies its rule-based logic to determine if a state change is needed.
        # This is the "brain" of our simple agent.

        if current_light_state == LIGHT_RED and time_in_current_state >=

```

DURATION_RED:

Rule: If currently RED and duration met, transition to GREEN.

current_light_state = LIGHT_GREEN

time_in_current_state = 0 # Reset timer for the new state

print(f"\n--- State Change Detected: Transitioning to {LIGHT_GREEN} ---\n")

elif current_light_state == LIGHT_GREEN and time_in_current_state >=

DURATION_GREEN:

Rule: If currently GREEN and duration met, transition to YELLOW.

current_light_state = LIGHT_YELLOW

time_in_current_state = 0 # Reset timer for the new state

print(f"\n--- State Change Detected: Transitioning to {LIGHT_YELLOW} ---\n")

elif current_light_state == LIGHT_YELLOW and time_in_current_state >=

DURATION_YELLOW:

Rule: If currently YELLOW and duration met, transition to RED.

current_light_state = LIGHT_RED

time_in_current_state = 0 # Reset timer for the new state

print(f"\n--- State Change Detected: Transitioning to {LIGHT_RED} ---\n")

3. Act (Simulated Actuator):

The agent "acts" by displaying its current light state and the time spent in it.

In a real system, this would involve sending signals to physical lights.

print(f"Traffic Light: {current_light_state} | Time in State:
{time_in_current_state}s")

```

    # Simulate time passing in the environment.

    # This pause makes the changes visible in real-time.

    time.sleep(1)

# --- Entry point to run the agent ---

if __name__ == "__main__":
    try:
        run_traffic_light_agent()

    except KeyboardInterrupt:
        # Allows graceful exit from the loop by pressing Ctrl+C

        print("\n--- Traffic Light Agent Stopped ---")

    except Exception as e:
        print(f"An error occurred: {e}")

```

Conclusion & Key Learning Points:

Congratulations! You've just built your very first AI agent blueprint. Even though it's simple, this Traffic Light Agent demonstrates several fundamental concepts:

- **States and Transitions:** How a system can move between defined states based on rules.
- **Rule-Based Logic:** The power of if-elif-else statements to dictate behavior.
- **The Agentic Cycle:** You saw the "perceive" (time increment), "decide" (rule application), and "act" (printing state, changing light) cycle in continuous operation.
- **Autonomy (Simple Form):** Once started, the agent runs independently without constant human intervention, following its programmed rules.

This basic state machine is the foundation upon which more complex, intelligent agents are built. As we move forward, we'll replace these simple rules with more sophisticated decision-making processes, often powered by Generative AI models.

Your Next Steps / Homework:

To solidify your understanding and get ready for our next sessions:

1. **Run the Code:** Experiment with the `traffic_light_agent.py` code. Observe how it cycles through the lights.
2. **Modify Durations:** Change the `DURATION_RED`, `DURATION_GREEN`, and `DURATION_YELLOW` values. How does the behavior change?
3. **Add a New State (Challenge!):** Can you add a new state, like `FLASHING_RED`, that the light might enter under certain conditions (e.g., after the `YELLOW` light, before going to `GREEN`, or maybe in an "emergency" scenario)? Think about what conditions would trigger this new state and how it would transition out.
4. **Think Beyond Time:** Right now, our agent only perceives time. How might you introduce a *different* kind of "sensor" to make it more intelligent? For example, if a real traffic light had a **car sensor** (simulated by a random event) that detected a car waiting, how might that change its decision-making logic? (Don't code this yet, just think about it!).

Get ready for our next session, where we'll explore how Generative AI, specifically Large Language Models, can become the "brain" of even more capable agents!